CENTRAL ASIAN UNIVERSITY
ENGINEERING
SCHOOL

CENTRAL ASIAN UNIVERSITY
School of Engineering
Computer Science Department

Journal of Internship for Graduation Project and Internship Module

Academic Year 2025-26

DIARY

(for the 4th year students)

Name: Mardonali Fayzullayev

ID: 230423

Supervisor(s) of practice: _____

Instructor: Professor Kiani Behnam

Year: 2026

Place of practice: Central Asian University (Online - You Tube)

Duration: 30 hours (1-2 weeks, flexible)

Module Code: Digital Image Processing (DIP)

Dates of internship

« 21 ***»*** April ***20***26 - ***«*** 02 ***»*** May ***20***26

Instruction of students on familiarization with the requirements of labor protection, safety equipment, fire safety passed:

Practice leader from a specialized

organization _____ ***(signature)***
Student _____ ***(signature)***

Here: 

Table of Contents

Introduction 4
Learning Outcomes of the Internship 5
Content of the Practice (According to the Profile of the Specialization) 5
Requirements for Educational and Methodological Support of Practice: 7
Supervisor (Definition) 9
Annex 1: Design Example - Title Page of Practice Diary 10
Week 1 Report 11
Week 2 Report 13
Daily Activities Summary 15

Introduction

Objectives of the Practice

During this short-term internship (2 weeks, 30 hours), students are expected to engage in practical and flexible learning activities based on a competency-oriented approach. The internship may include workplace experience, faculty-assigned tasks, or approved online learning modules.

1. **Application of Academic Knowledge**
    - Apply theoretical concepts learned in coursework to practical or real-world tasks.
    - Develop problem-solving skills relevant to the student's field of study.
2. **Exposure to Professional or Academic Work Environments**
    - Gain familiarity with professional practices, workflows, or research-oriented tasks.
    - Understand basic organizational structures and work processes.
3. **Development of Technical and Analytical Skills**
    - Perform basic technical, analytical, or design-related tasks depending on the specialization.
    - Engage in activities such as data handling, system understanding, simulations, or tool usage.
4. **Independent and Guided Learning**
    - Complete assigned tasks independently or under supervision (e.g., employer, professor, or online platform).
    - Follow structured learning through modules, tutorials, or project-based work.
5. **Communication and Collaboration**
    - Demonstrate basic communication skills through reporting, discussions, or task updates.
    - Collaborate with peers, supervisors, or mentors when applicable.
6. **Reflection and Reporting**
    - Maintain a simple record of activities and learning outcomes.
    - Prepare a brief internship summary highlighting key tasks and skills developed.

The Number of Hours for Mastering Work Programs:

Duration: 1–2 weeks
Total Workload: 30 hours (flexible distribution)

Learning Outcomes of the Internship

By the end of the internship (2 weeks, 30 hours), students are expected to achieve the following:

- Understand how basic tasks or projects are planned and completed
- Complete assigned tasks independently or with guidance
- Show interest in their field of study and professional development
- Manage their time and complete tasks responsibly
- Solve simple problems and make basic decisions
- Find and use information (online or offline) for their tasks
- Use basic digital tools and technologies related to their field
- Communicate clearly with supervisors, professors, or teammates
- Work individually or as part of a team
- Reflect on their learning and identify areas for improvement

Content of the Practice (According to the Profile of the Specialization)

During the internship (2 weeks, 30 hours), students can complete **any combination of the following activities** depending on their situation (job, university tasks, or online learning):

1. Getting Started (2–3 hours)

- Understand internship goals and requirements
- Learn basic safety and workplace/online rules
- Get familiar with the organization, project, or assigned tasks

2. Main Activities (20–24 hours)

Students may choose or be assigned tasks such as:

- Working on small technical or engineering-related tasks
- Assisting in projects at workplace or university
- Completing online courses or tutorials (approved by instructor)
- Practicing programming, data analysis, design, or simulations
- Learning tools, software, or platforms related to their field
- Reading and understanding technical materials

(Tasks can be individual or supervised by a mentor/professor.

3. Basic Skills Development

During the internship, students should try to:

- Solve simple practical problems
- Learn or improve technical skills
- Work independently or with guidance
- Communicate progress when required

4. Optional Area-Based Activities (if relevant)

Students may focus on one area depending on their interest:

- **Technical / Software:** simple coding, testing, or tool usage
- **Data / Analysis:** basic data handling, visualization, or reporting
- **Systems / Networks:** setup, configuration, or troubleshooting
- **Laboratory / Equipment:** experiments, simulations, or technical measurements
- **Design / Projects:** drafting, modeling, or process improvement tasks

(Students are NOT required to cover all areas.)

5. Final Submission (3–5 hours)

- Fill in a **simple daily diary/log**
- Write a report in mentioned format about
    - What was done
    - What was learned

- Complete **30 hours** of internship activities
- Submit a **diary/report in Word or pdf format**

No printing or hard copy is required.

Note

- Internship can be completed through:
    - Full-time or part-time job
    - Tasks assigned by a professor
    - Approved online courses/platforms

Requirements for Educational and Methodological Support of Practice:

What Students Need to Submit

- A **simple daily diary/log** (short notes of what you did)
- A **short report (1–2 pages)** about your experience
- (Optional) Any **supporting work** (tasks, screenshots, certificates, etc.)

If you are working or doing an online course, proof (email, certificate, or confirmation) is enough. No stamps/seals required.

Internship Format

The internship can be completed through:

- A job (full-time or part-time)
- Tasks assigned by a professor
- Online courses or self-learning (approved by module instructor)

Evaluation Criteria

The internship is evaluated as part of the related course/module (e.g., Image Processing, Software Engineering, etc.) and carries **10 marks**.

- **Completion of required hours (30 hours)** – required
- **Submission of diary and short report** – required

Grading (Total: 10 Marks)

- Completion of internship requirements – **Pass/Fail**
- Based on completion, students will be awarded up to **10 marks** in the relevant module

Students who do not complete the requirements will not receive marks.

Support from University

- Simple diary/report templates will be provided
- Guidance from assigned professors (if needed)

Contact (if needed)

- Module instructor
- Head of Department

Main Sources:

- Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall 2008.
- Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley 2018.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*. MIT Press 2009.
- Steve McConnell. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press 2004.
- Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*. Pearson 2014.
- Ian Sommerville. *Software Engineering*. Pearson 2015.
- Raj Jain. *The Art of Computer Systems Performance Analysis*. Wiley 1991.

Internet Resources:

- Stack Overflow
- GitHub
- Coursera
- Khan Academy
- edX

Supervisor (Definition)

A **Supervisor** is the person who oversees or confirms the student's internship activities. This can be:

- A **workplace manager or team lead** (for students doing a job)
- A **university professor or instructor of relevant module** (for assigned academic tasks or online modules)

The supervisor is responsible for:

- Confirming that the student has completed the internship activities
- Providing simple approval (name or digital signature)

Only one supervisor is required.

Annex 1: Design Example - Title Page of Practice Diary

Department of Computer Science

DIARY

Practice for Obtaining Professional Skills and Professional Experience

FINAL SUMMARY

Student Name:	Mardonali Fayzullayev
Student ID:	230423
Module:	Digital Image Processing
Internship Period:	April 21, 2025 — May 2, 2025
Total Days:	12 days
Daily Hours:	2.5 hours
Total Hours Completed:	30 hours
Learning Source:	YouTube — sentdex (OpenCV with Python Tutorial Series)
Topics Covered:	14 topics across 2 weeks (color spaces, filtering, thresholding, edge detection, contours, morphology, histograms, template matching, face detection)
Final Project:	Real-time face detection application using OpenCV and webcam
Supervisor:	Professor Kiani Behnam

Hello Professor, This is my own work, I wrote it myself without any AI

Week 1 Report

Plans (To be defined after discussing with the supervisor)

- Plan 1

Learn the fundamentals of the OpenCV library in Python:
setting up the development environment, installing OpenCV via pip,
understanding how images are stored as NumPy arrays, and practicing core functions such as cv2.imread(), cv2.imshow(), cv2.imwrite(), and cv2.waitKey() to read, display, and save images in different formats.

- Plan 2

Study color space theory and conversions between BGR, Grayscale, and HSV using cv2.cvtColor();
practice image resizing with cv2.resize() using different interpolation methods;
learn region-of-interest cropping via NumPy slicing;
draw shapes and annotations using cv2.line(), cv2.rectangle(), cv2.circle(), and cv2.putText();
apply binary and adaptive thresholding using cv2.threshold() and cv2.adaptiveThreshold() including Otsu's method;
reduce image noise using smoothing filters such as cv2.blur(), cv2.GaussianBlur(), cv2.medianBlur(),
and finally cv2.bilateralFilter().

- Plan 3

Consolidate all Week 1 techniques by building a complete image preprocessing pipeline that chains grayscale conversion,
Gaussian blur, and Otsu's thresholding together;
test the pipeline on multiple images of different types;
write clean, well-commented Python code and verify results at each stage of the pipeline.

Achievements **(Describe what tools/languages you explored, what new thing you learnt, how are you going to use them in future)**

During Week 1, I followed the sentdex YouTube tutorial series on OpenCV with Python and successfully completed all planned topics. On Day 1, I set up my Python environment, installed OpenCV using pip, and wrote my first image processing scripts. I learned that OpenCV stores images as multi-dimensional NumPy arrays in BGR channel order, and practiced reading images from disk using cv2.imread(), displaying them in a window with cv2.imshow(), and saving processed results using cv2.imwrite().

On Day 2, I studied color spaces in depth. I used cv2.cvtColor() to convert images between BGR, Grayscale, and HSV formats, and applied cv2.inRange() to isolate specific color ranges in HSV space, which is useful for object tracking. I compared the visual results of each conversion to understand when each color space is most appropriate.

On Day 3, I practiced image resizing using cv2.resize() with INTER_LINEAR and INTER_CUBIC interpolation and compared their quality. I cropped regions of interest using NumPy array slicing and drew geometric annotations using cv2.line(), cv2.rectangle(), cv2.circle(), and cv2.putText(), which are essential for visualizing detection results.

On Day 4, I explored thresholding techniques. I applied simple binary thresholding, BINARY_INV, and TRUNC variants, then moved to cv2.adaptiveThreshold() which computes local thresholds and handles uneven lighting. I also applied Otsu's binarization which automatically finds the optimal threshold from the image histogram without manual tuning.

On Day 5, I studied four
types of smoothing filters: cv2.blur() for simple averaging, cv2.GaussianBlur() for weighted smoothing, cv2.medianBlur() for impulse noise removal, and cv2.bilateralFilter() for edge-preserving smoothing. I compared all four on the same noisy image with different kernel sizes.

To conclude Week 1, I built a complete image preprocessing pipeline that chains grayscale conversion, Gaussian blur, and Otsu's thresholding and tested it on five different images. I plan to use all of these foundational skills in Week 2 for more advanced image analysis and in future computer vision projects.

Week 2 Report

Plans (To be defined after discussing with the supervisor)

- Plan 1

Study edge detection theory and apply the Canny algorithm using cv2.Canny() with different threshold values to understand how sensitivity changes;
also apply the Sobel operator using cv2.Sobel() in both X and Y directions and combine the results to compute gradient magnitude; then practice contour detection using cv2.findContours() with different retrieval modes,
draw contours with cv2.drawContours(), and compute geometric properties such as area using cv2.contourArea(), perimeter using cv2.arcLength(), and bounding rectangles using cv2.boundingRect().

- Plan 2

Explore morphological image processing by creating custom structuring elements with cv2.getStructuringElement() and
applying erosion using cv2.erode(), dilation using cv2.dilate(),
morphological opening and closing using cv2.morphologyEx();
then study image histograms using cv2.calcHist() and visualize them with matplotlib; apply global histogram equalization using cv2.equalizeHist()
and compare it with CLAHE (Contrast Limited Adaptive Histogram Equalization) using cv2.createCLAHE() on images with different lighting conditions.

- Plan 3

Implement template matching using cv2.matchTemplate() with multiple matching methods including TM_CCOEFF_NORMED and TM_SQDIFF_NORMED,
locate the best match using cv2.minMaxLoc(), and draw a bounding box around the result;

then apply a Haar Cascade classifier using cv2.CascadeClassifier() and detectMultiScale() to detect faces in still images;

complete the internship with a final real-time project: a webcam-based face detection application using cv2.VideoCapture() that processes live video frames and draws annotated bounding boxes around detected faces.

Achievements **(Describe what tools/languages you explored, what new thing you learnt, how are you going to use them in future)**

During Week 2, I advanced to higher-level image processing and computer vision techniques, building on all the foundational skills developed in Week 1.

On Day 7, I studied the Canny edge detection algorithm in detail, learning how it works in four stages: Gaussian smoothing, gradient computation using Sobel filters, non-maximum suppression to thin edges, and hysteresis thresholding to retain only strong, connected edges. I applied cv2.Canny() with different low and high threshold values and compared results. I also manually computed edges using cv2.Sobel() in X and Y directions and combined them with cv2.addWeighted().

On Day 8, I practiced contour detection using cv2.findContours() with RETR_EXTERNAL and RETR_TREE modes and CHAIN_APPROX_SIMPLE approximation. I drew contours using cv2.drawContours() and computed geometric descriptors: area with cv2.contourArea(), perimeter with cv2.arcLength(), and bounding rectangles with cv2.boundingRect(). I also used cv2.approxPolyDP() to approximate contour shapes and filtered contours by minimum area to remove noise.

On Day 9, I studied morphological operations. I created structuring elements of different shapes using cv2.getStructuringElement() and applied cv2.erode() to shrink regions, cv2.dilate() to expand them, and cv2.morphologyEx() with MORPH_OPEN and MORPH_CLOSE to remove noise and fill holes in binary images. I tested all operations on thresholded text images.

On Day 10, I computed and plotted image histograms using cv2.calcHist() and matplotlib. I applied global histogram equalization with cv2.equalizeHist() and compared it with CLAHE using cv2.createCLAHE(), which applies equalization locally with a clip limit and produces more natural contrast enhancement. I tested both on images with poor lighting.

On Day 11, I implemented template matching using cv2.matchTemplate() with TM_CCOEFF_NORMED and found the best match location using cv2.minMaxLoc(). I also applied the built-in Haar Cascade frontal face classifier using cv2.CascadeClassifier() and detectMultiScale() to detect faces in still images with bounding boxes.
On Day 12, I completed the internship with a real-time face detection project. I used cv2.VideoCapture(0) to open the webcam, read frames in a loop, converted each frame to grayscale, applied the Haar Cascade detector, and drew bounding boxes around detected faces using cv2.rectangle(). I tuned scaleFactor and minNeighbors parameters to reduce false positives and achieve stable detection. This final project successfully combined every concept from both weeks into a working computer vision application.

Daily Activities Summary

Week 1

Day	Module covered	Supervisor's remarks
1 (April 21, 2026 — 2.5 hours)	Set up Python environment and installed OpenCV using pip. Studied how images are stored as NumPy arrays in memory. Practiced cv2.imread() to load images in color and grayscale modes, cv2.imshow() to display images in a window, cv2.waitKey() to control window timing.	
2 (April 22, 2026 — 2.5 hours)	Studied color space theory and conversions. Used cv2.cvtColor() to convert images between BGR, Grayscale, and HSV color spaces and displayed each result side by side for visual comparison.	
3 (April 23, 2026 — 2.5 hours)	Practiced image resizing using cv2.resize() with INTER_LINEAR, INTER_NEAREST, and INTER_CUBIC interpolation methods and compared visual quality when upscaling and downscaling. Cropped specific regions of interest using NumPy array slicing with row and column index ranges.	

4 April 24	Binary, adaptive thresholding and Otsu's binarization: cv2.threshold(), cv2.adaptiveThreshold()	
5 April 25-26	Smoothing filters: cv2.blur(), cv2.GaussianBlur(), cv2.medianBlur(), cv2.bilateralFilter(); mini pipeline project	

Supervisor's signature (Digital) _____

Week 2

Day	Module covered	Supervisor's remarks
1 April 27	Edge detection: Canny algorithm and Sobel operator; gradient magnitude computation	
2 April 28	Contour detection with cv2.findContours(); area, perimeter and bounding box computation	
3 April 29	Morphological operations: erosion, dilation, opening and closing with custom structuring elements	

4 April 30	Image histograms with cv2.calcHist(), global histogram equalization and CLAHE	
5 May 1-2	Template matching; Haar Cascade face detection; final real-time webcam face detection project using cv2.VideoCapture()	

Supervisor's signature (Digital) _____